Introduction

Welcome to the Text Normalization assignment! In the realm of Natural Language Processing (NLP), ensuring that textual data is in a consistent and standardized format is crucial for effective analysis and modeling. Text normalization involves a series of preprocessing tasks aimed at enhancing the quality and uniformity of textual data.

In this assignment, we will delve into text normalization, exploring various techniques and algorithms to transform raw text into a more structured and analyzable form. Our primary focus will be on understanding and implementing the Byte Pair Encoding (BPE) algorithm, a popular method for subword tokenization.

This assignment is designed to deepen your knowledge and proficiency in text normalization. As you progress through the tasks, consider the impact of each normalization technique on downstream NLP tasks and develop an appreciation for the subtleties involved in ensuring text quality and consistency.

Let's embark on this exciting exploration of text normalization, where precision meets language, and consistency meets comprehension!

Byte Pair Encoding (BPE) algorithm

In [22]:
```
import string
```

For the tasks in this assignment, we will be working with the renowned play "Hamlet" by William Shakespeare. "Hamlet" is a masterpiece of English literature, celebrated for its profound themes, intricate characters, and rich linguistic tapestry. Run this cell to load the "Hamlet" text data.

In [23]:
```
def read_data(file_path):
    # Open the file in read mode ('r')
    with open(file_path, 'r') as file:
        # Read the entire file content
        return file.read()

text = read_data(r'C:\Users\hasan\Downloads\3.2C-resources\3.2C - BPE Algorithm\data\hamlet.txt')
```

We now prepare our text by removing punctuation and converting to lower case:

In [24]:
```
def clean_and_normalize_text(text):
    """
    Clean and normalize the input text by converting it to lowercase
    and removing punctuation.

    Parameters:
    - text (str): The input text to be cleaned.

    Returns:
    - str: The cleaned and normalized text.
    """
    text = text.lower()
    translator = str.maketrans("", "", string.punctuation)
    cleaned_text = text.translate(translator)
    return cleaned_text

cleaned_text = clean_and_normalize_text(text)
```

Task 1

Create a function that computes token frequencies and establishes the initial vocabulary:

In [25]:
```
def calculate_token_frequencies_and_vocabulary(text):
    """
    Calculate token frequencies and build a vocabulary from the input text.

    Parameters:
    - text (str): The input text.

    Returns:
    - tuple: A tuple containing a dictionary of token frequencies and a set of vocabulary.
    """
    # Create a dictionary to store token frequencies
    token_frequency = {}
    vocabulary = set()
    ## START YOU CODE HERE
    words = text.split()

    for word in words:
```

```
        # Convert word into BPE format (characters + end symbol)
        token = " ".join(list(word)) + " _"

        # Count frequency
        if token in token_frequency:
            token_frequency[token] += 1
        else:
            token_frequency[token] = 1

        # Add characters to vocabulary
        for char in word:
            vocabulary.add(char)

    # Add end-of-word symbol
    vocabulary.add("_")
     ## END
    return token_frequency, vocabulary

data, vocabulary = calculate_token_frequencies_and_vocabulary(cleaned_text)
```

Run the below cell to test your function:

In [26]:
```
test_data, test_vocabulary = calculate_token_frequencies_and_vocabulary("low low low low low lowest lowest newer newer new
assert test_data == {'l o w _': 5, 'l o w e s t _': 2, 'n e w e r _': 6, 'w i d e r _': 3, 'n e w _': 2}, "Test failed!"
assert test_vocabulary == {'_', 'd', 'e', 'i', 'l', 'n', 'o', 'r', 's', 't', 'w'}, "Test failed!"
print("Success!")
```

```
Success!
```

Task 2

Create a function that identifies the most frequent pair of adjacent symbols in the data:

In [27]:
```
def calculate_most_frequent_symbol_pair(data):
    """
    Calculate most frequent pair of adjacent symbols in the given data.

    Parameters:
    - data (dict): A dictionary where keys are words and values are their frequencies.

    Returns:
    - tuple: A tuple containing the most frequent pair of symbols and its frequency.
    """
    best_pair = ()
    frequency = 0
    ## START YOU CODE HERE
    pair_counts = {}

    for word, freq in data.items():
        symbols = word.split()

        for i in range(len(symbols) - 1):
            pair = (symbols[i], symbols[i+1])

            if pair in pair_counts:
                pair_counts[pair] += freq
            else:
                pair_counts[pair] = freq

    # Find the most frequent pair
    for pair, freq in pair_counts.items():
        if freq > frequency:
            best_pair = pair
            frequency = freq
    ## END
    return best_pair, frequency
```

Run the below cell to test your function:

In [28]:
```
assert calculate_most_frequent_symbol_pair(test_data) == (('e', 'r'), 9), "Test failed!"
print("Success!")
```

```
Success!
```

Task 3

Create a function that merges a given symbol and update the data and vocabulary:

In [29]:
```
def merge_symbol(pair, data, vocabulary):
    """
    Merge the specified symbol pair into a single symbol and update the data and vocabulary.

    Parameters:
```

```
    - pair (tuple): The symbol pair to be merged.
    - data (dict): A dictionary where keys are words, and values are their frequencies.
    - vocabulary (set): A set containing the vocabulary of symbols.

    Returns:
    - tuple: A tuple containing the updated data, updated vocabulary, and the newly created symbol.
    """
    # Merge occurrences of the pair into a single symbol
    updated_data = {}
    ## START YOU CODE HERE
    new_symbol = pair[0] + pair[1]

    for word, freq in data.items():
        symbols = word.split()
        i = 0
        new_symbols = []

        while i < len(symbols):
            # Check if pair matches
            if i < len(symbols) - 1 and symbols[i] == pair[0] and symbols[i+1] == pair[1]:
                new_symbols.append(new_symbol)
                i += 2  # skip next (merged)
            else:
                new_symbols.append(symbols[i])
                i += 1

        updated_word = " ".join(new_symbols)
        updated_data[updated_word] = freq

    updated_vocabulary = set(vocabulary)
    updated_vocabulary.add(new_symbol)
    ## END
    return updated_data, updated_vocabulary, new_symbol
```

Run the below cell to test your function:

In [30]:
```
assert merge_symbol(('e', 'r'), test_data, test_vocabulary) == ({'l o w _': 5, 'l o w e s t _': 2, 'n e w er _': 6,
                                                                 'w i d er _': 3, 'n e w _': 2},
                                                                {'_', 'd', 'e', 'er', 'i', 'l', 'n', 'o', 'r', 's', 't', '
                                                                'er'), "Test failed!"

print("Success!")
assert merge_symbol(('e', 'r'), {'de ri_': 5, 'n e w e r _': 6},
                    {'_', 'd', 'e', 'i', 'n', 'r',}) == ({'de ri_': 5, 'n e w er _': 6},
                                                         {'_', 'd', 'e', 'er', 'i', 'n', 'r'}, 'er'), "Test failed!"
print("Success!")
```

```
Success!
Success!
```

Task 4

Create the BPE function that will run a number of merges:

In [31]:
```
def bpe(data, vocabulary, num_merges):
    """
    Apply Byte Pair Encoding (BPE) to the given data for a specified number of merges.

    Parameters:
    - data (dict): A dictionary where keys are words and values are their frequencies.
    - vocabulary (set): A set containing the initial vocabulary of symbols.
    - num_merges (int): The number of merges to perform.

    Returns:
    - tuple: A tuple containing the updated data, updated vocabulary, and a list of merge rules (ranked).
    """
    merge_rules = []
    updated_data = data
    updated_vocabulary = vocabulary
    ## START YOU CODE HERE
    for _ in range(num_merges):
        # Step 1: find most frequent pair
        pair, freq = calculate_most_frequent_symbol_pair(updated_data)

        if not pair:
            break

        # Step 2: merge
        updated_data, updated_vocabulary, new_symbol = merge_symbol(
            pair, updated_data, updated_vocabulary
        )

        # Step 3: store rule in correct format
        merge_rules.append((" ".join(pair), new_symbol))
```

```
    ## END
    return updated_data, updated_vocabulary, merge_rules
```

In [32]:
```
updated_test_data, updated_test_vocabulary, test_merge_rules = bpe(test_data, test_vocabulary, 8)
assert updated_test_data == {'low_': 5, 'low e s t _': 2, 'newer_': 6, 'w i d er_': 3, 'new _': 2}, "Test failed!"
assert updated_test_vocabulary == {'_','d','e','er','er_','i','l','lo','low','low_','n','ne','new','newer_','o','r','s','t
assert test_merge_rules == [('e r', 'er'), ('er _', 'er_'), ('n e', 'ne'), ('ne w', 'new'),
                            ('l o', 'lo'), ('lo w', 'low'), ('new er_', 'newer_'), ('low _', 'low_')]
print("Success!")
```

```
Success!
```

Training

Let's now learn our merge rules on the Hamlet text data.

In [33]:
```
updated_data, updated_vocabulary, merge_rules = bpe(data, vocabulary, 400)
```

Tokenization

Let's construct our tokenizer, leveraging the merge rules acquired in the previous section.

Task 5

Write a function that uses the merge_rules to tokenize a text.

In [34]:
```
def token_segmenter(text, merge_rules):
    """
    Tokenize the input text using BPE and the merge rules.

    Parameters:
    - text (str): The input text to be tokenized.
    - merge_rules (list): A list of merging rules obtained from the BPE algorithm.

    Returns:
    - list: A list of tokens obtained using token segmentation rules.
    """
    text = clean_and_normalize_text(text)
    ## START YOU CODE HERE
    words = text.split()
    tokenized_text = []

    for word in words:
        # Step 1: convert to symbols
        symbols = list(word) + ['_']

        # Step 2: apply merge rules one by one
        for pair_str, merged in merge_rules:
            pair = pair_str.split()  # ['e','r']

            i = 0
            new_symbols = []

            while i < len(symbols):
                # check if pair matches
                if i < len(symbols)-1 and symbols[i] == pair[0] and symbols[i+1] == pair[1]:
                    new_symbols.append(merged)
                    i += 2
                else:
                    new_symbols.append(symbols[i])
                    i += 1

            symbols = new_symbols

        # Step 3: collect tokens
        tokenized_text.extend(symbols)
    ## END
    return tokenized_text
```

In [35]:
```
tokenized_text = token_segmenter("that which we call a rose by any other name would smell as sweet", merge_rules)
assert tokenized_text == ['that_', 'which_', 'we_', 'ca', 'll_', 'a_', 'ro',
                          'se_', 'by_', 'an', 'y_', 'other_', 'n', 'a', 'me_',
                          'would_', 's', 'm', 'ell_', 'as_', 'sw', 'eet_'], "Test failed!"
print("Success!")
```

```
Success!
```

Congratulations!

Congratulations on completing the assignment! Your dedication and effort are commendable. By successfully working through the coding exercises and written exercises, you have demonstrated a strong understanding of the concepts and principles related to text normalization and tokenization.

Congratulations on finishing this notebook!

Acknowledgement

About the Author

This notebook was authored by Mohamed Reda Bouadjenek. He is a Senior Lecturer (Assistant Professor) of Applied Artificial Intelligence in the School of Information Technology at Deakin University, Australia.

Contact Information

- **Email:** reda.bouadjenek@deakin.edu.au
- **GitHub:** https://github.com/rbouadjenek/